



2η Σειρά Ασκήσεων

Ονοματεπώνυμο Α.Μ.

Μαλαμάς Νικόλαος: 2020030180

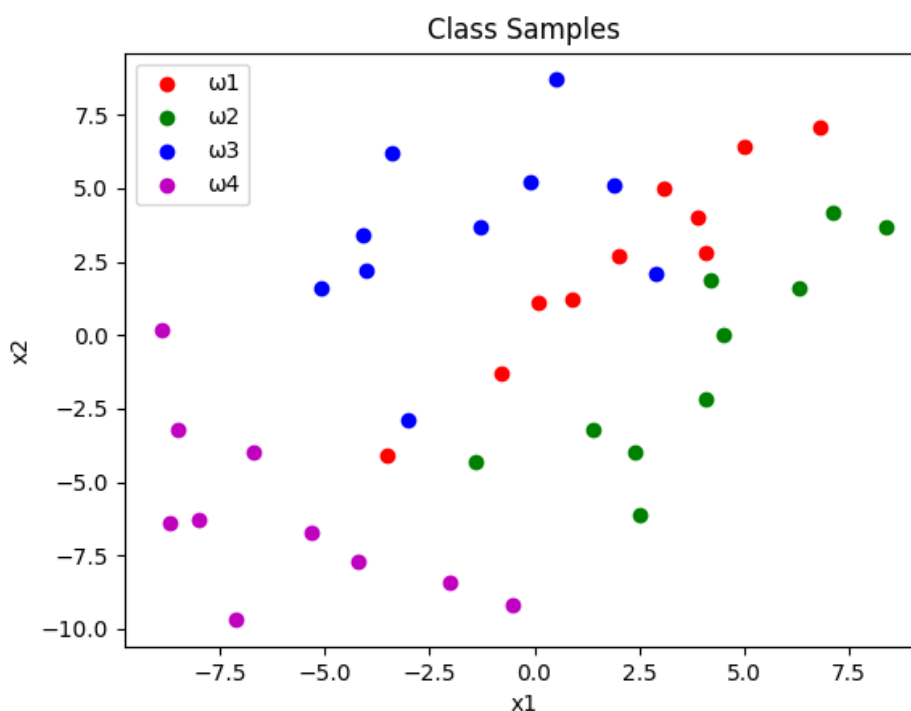
Πατσέα Ιωάννα-Γεωργία: 2020030033

Ομάδα 11

Θέμα 1:Αλγόριθμος Perceptron

Σε αυτή την άσκηση χρησιμοποιήθηκε ο αλγόριθμος Perceptron.

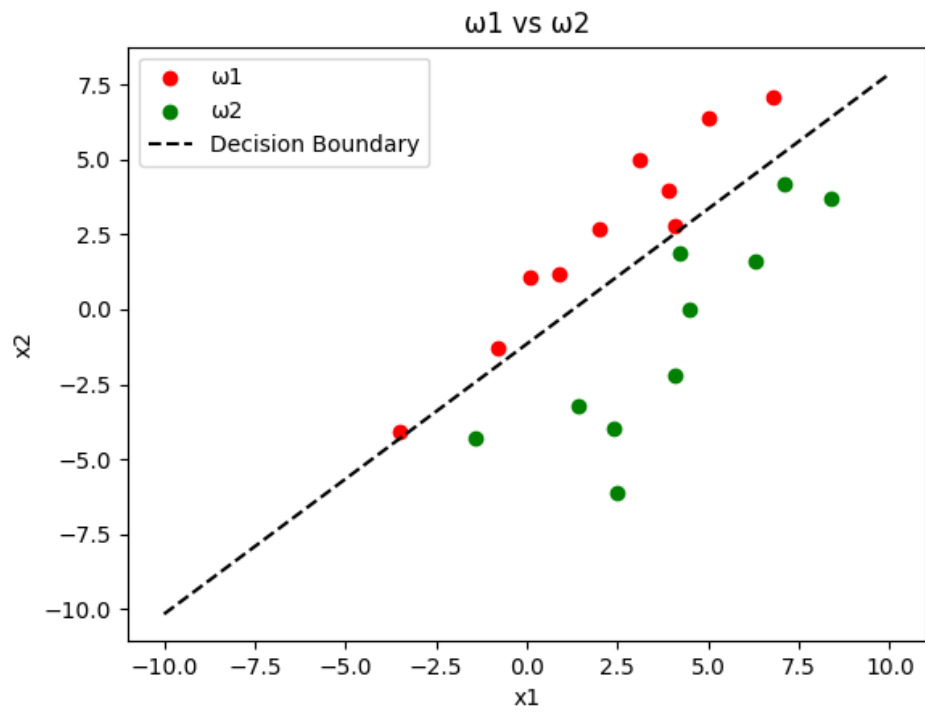
α') Το κοινό διάγραμμα των κλάσεων επισυνάπτεται παρακάτω:



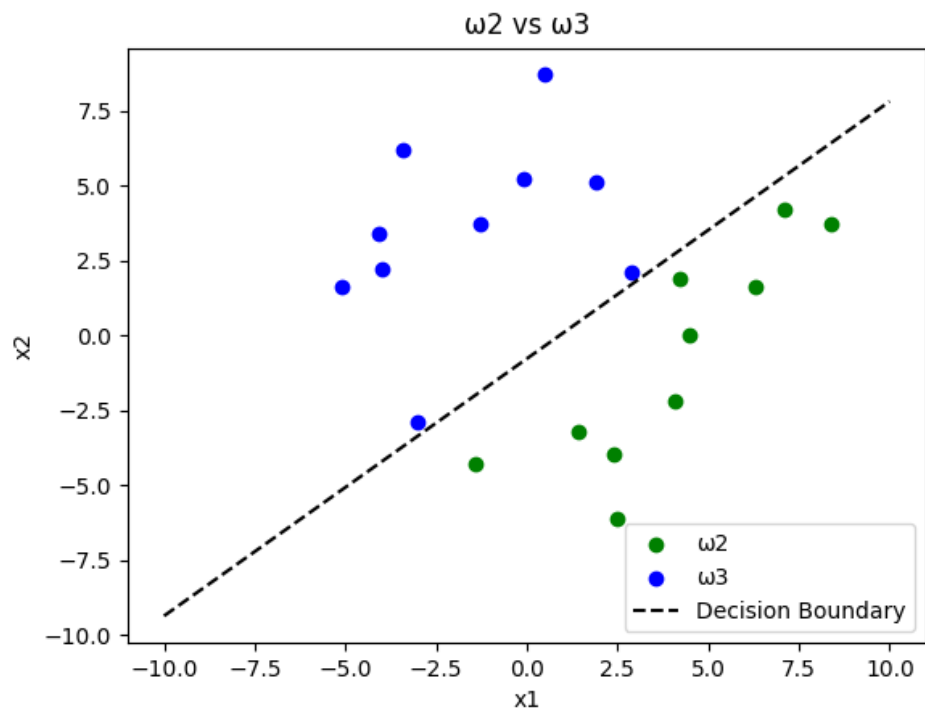
Σχήμα 1: Το κοινό διάγραμμα κλάσεων

β') Όπως καθορίζει η εκφώνηση, το διάνυσμα βαρών αρχικοποιείται σε μηδέν και καταγράφεται ο αριθμός των επαναλήψεων έως ότου συγκλίνει ο αλγόριθμος

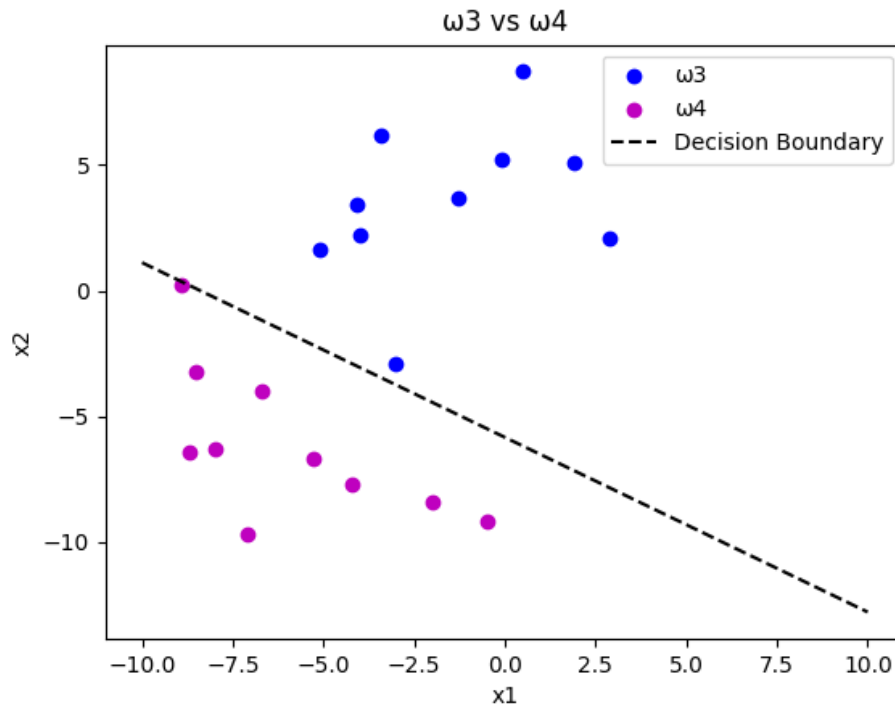
γ') Παρακάτω επισυνάπτονται τα κοινά διαγράμματα μεταξύ διαφορετικών ζευγαριών κλάσεων με το αντίστοιχο όριο απόφασης και ο αριθμός επαναλήψεων που χρειάστηκε ο αλγόριθμος για να συγκλίνει σε κάθε περίπτωση.



Σχήμα 2: Το όριο απόφασης μεταξύ w_1 και w_2



Σχήμα 3: Το όριο απόφασης μεταξύ w_2 και w_3



Σχήμα 4: Το όριο απόφασης μεταξύ w_3 και w_4

Στην κονσόλα εμφανίζονται τα εξής αποτελέσματα αναφορικά με τις επαναλήψεις σύγκλισης:

```
Number of updates for w1 vs w2: 33
Number of updates for w2 vs w3: 9
Number of updates for w3 vs w4: 53
```

Η παρατηρούμενη διαφορά στον αριθμό επαναλήψεων για σύγκλιση του αλγορίθμου, για διαφορετικά ζευγάρια κλάσεων, οφείλεται στο ότι τα δυσδιάστατα δείγματα ορισμένων κλάσεων τοποθετούνται αρκετά κοντά το ένα με το άλλο στον 2D χώρο με αποτέλεσμα ο αλγόριθμος να χρειάζεται πολλές επαναλήψεις για τον διαχωρισμό του (πχ w_1 vs w_2).

Θέμα 2: Λογιστική Παλινδρόμηση: Αναλυτική εύρεση κλίσης (Gradient)

Σε αυτή την άσκηση θα γίνει προσπάθεια πρόβλεψης σχετικά με την απορρόφιση ή αποδοχή μαθητών σύμφωνα με δύο test που αυτοί ολοκλήρωσαν, χρήσει Linear Regression.

Απόδειξη του (d):

Η συνάρτηση κόστους είναι:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [-y^{(i)} \ln(\hat{y}^{(i)}) - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})]$$

όπου $\hat{y}^{(i)} = h_{\theta}(x^{(i)}) = f(\theta^T x^{(i)})$.

Η παράγωγος της $J(\theta)$ ως προς θ_j :

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \left[\frac{1}{m} \sum_{i=1}^m (-y^{(i)} \ln(\hat{y}^{(i)}) - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})) \right]$$

Αρχικά υπολογίζεται η παράγωγος του αθροίσματος:

$$\frac{\partial}{\partial \theta_j} (-y^{(i)} \ln(\hat{y}^{(i)}) - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}))$$

Παράγωγος του $\ln(\hat{y}^{(i)})$:

$$\frac{\partial}{\partial \theta_j} \ln(\hat{y}^{(i)}) = \frac{1}{\hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial \theta_j}$$

Επίσης, έχουμε:

$$\hat{y}^{(i)} = f(\theta^T x^{(i)}) = \frac{1}{1 + 2e^{-\theta^T x^{(i)}}}$$

Υπολίζοντας την παράγωγο του $\hat{y}^{(i)}$:

$$\frac{\partial \hat{y}^{(i)}}{\partial \theta_j} = \hat{y}^{(i)} (1 - \hat{y}^{(i)}) x_j^{(i)}$$

Συνεπώς:

$$\frac{\partial}{\partial \theta_j} \ln(\hat{y}^{(i)}) = \frac{1}{\hat{y}^{(i)}} \cdot \hat{y}^{(i)} (1 - \hat{y}^{(i)}) x_j^{(i)} = (1 - \hat{y}^{(i)}) x_j^{(i)}$$

Ακολουθούμε την ίδια διαδικασία για το $\ln(1 - \hat{y}^{(i)})$:

$$\frac{\partial}{\partial \theta_j} \ln(1 - \hat{y}^{(i)}) = \frac{1}{1 - \hat{y}^{(i)}} \cdot (-\hat{y}^{(i)}) (1 - \hat{y}^{(i)}) x_j^{(i)} = -\hat{y}^{(i)} x_j^{(i)}$$

Συνοψίζοντας τα παραπάνω, έχουμε:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} (-y^{(i)} \ln(\hat{y}^{(i)}) - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})) &= -y^{(i)} (1 - \hat{y}^{(i)}) x_j^{(i)} + (1 - y^{(i)}) \hat{y}^{(i)} x_j^{(i)} \\ &= x_j^{(i)} [-y^{(i)} (1 - \hat{y}^{(i)}) + (1 - y^{(i)}) \hat{y}^{(i)}] \\ &= x_j^{(i)} [-y^{(i)} + y^{(i)} \hat{y}^{(i)} + \hat{y}^{(i)} - y^{(i)} \hat{y}^{(i)}] \end{aligned}$$

$$= x_j^{(i)} [\hat{y}^{(i)} - y^{(i)}]$$

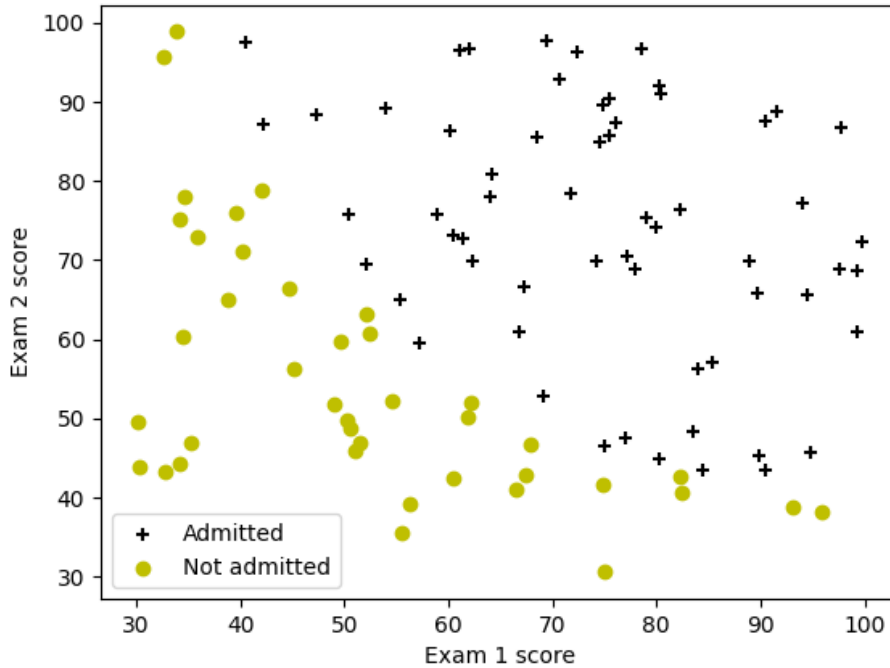
Το συνολικό άθροισμα γίνεται:

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_j} &= \frac{1}{m} \sum_{i=1}^m x_j^{(i)} (\hat{y}^{(i)} - y^{(i)}) \\ &= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \end{aligned}$$

Έτσι αποδεικνύεται ότι:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- i. Συμπληρώθηκε ο κώδικας python στο αρχείο που δώθηκε MyLogit.py. Χρησιμοποιήθηκαν μόνο τα modules(numpy, matplotlib, scipy.optimize) σε όλη την άσκηση.
- ii. Διαβάζοντας τα δείγματα από το αρχείο 'exam_scores_data1.txt' και σχεδιάζοντάς τα λαμβάνουμε τα παρακάτω:

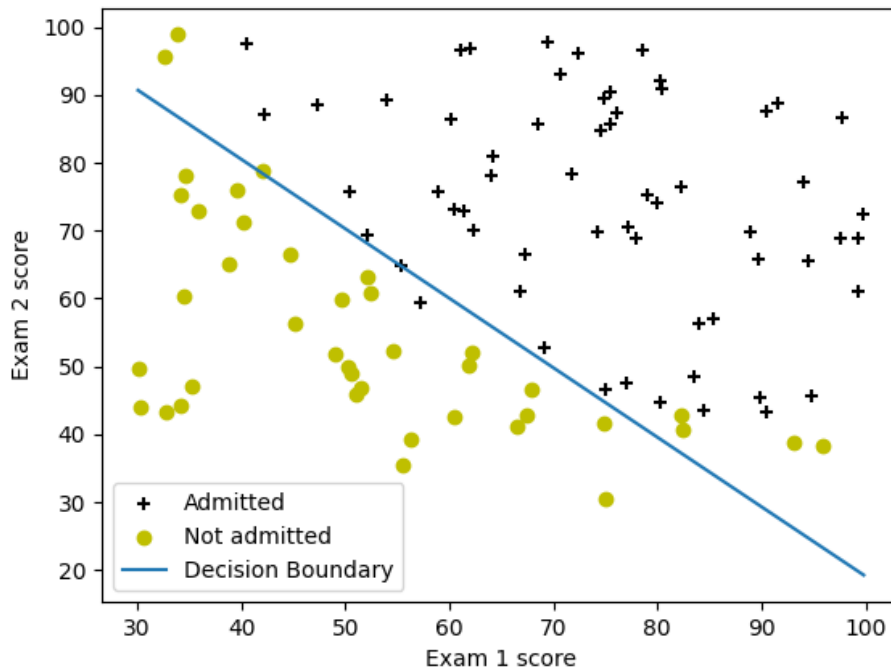


Σχήμα 5: Pass-Fail αποτελέσματα μεταξύ βαθμολογιών των δύο τεστ

- iii. Συμπληρώθηκε ο αντίστοιχος κώδικας της 'sigmoid()'.
- iv. Συμπληρώθηκε ο αντίστοιχος κώδικας για τις συναρτήσεις costFunction και gradient. Παρακάτω παρουσιάζεται το κόστος και η κλίση στο $\theta_0 = [0 \ 0 \ \dots \ 0]$ όπως αυτά εκτυπώθηκαν στην κονσόλα.

```
Cost at initial theta (zeros): 0.8213534164441316
Gradient at initial theta (zeros): [-0.26666667
                                     -22.94992893
                                     -22.29984189]
```

- v. Με την συμπλήρωση και την εκτέλεση του κώδικα λαμβάνουμε το σύνολο απόφασης που φαίνεται στο παρακάτω διάγραμμα:



Σχήμα 6: Decision Boundary

Μετά την ολοκλήρωση της βελτιστοποίησης παραμέτρων, λαμβάνουμε στην κονσόλα την τιμή του θ που δίνει τη βέλτιστη τιμή σφάλματος, καθώς και ποια είναι αυτή.

```
Cost at theta found by minimize: 0.20349770159239447
theta: [-24.46817402  0.20623133  0.2014718 ]
```

Στην συνέχεια υλοποιήθηκε η συνάρτηση 'predict()', υπεύθυνη για την πρόβλεψη απόφασης για την αποδοχή-απόρριψη του εκάστοτε μαθητή σύμφωνα με το σκορ του στα δύο τεστ. Για παράδειγμα, για έναν μαθητή με σκορ στο πρώτο τεστ 45 και 85 στο δεύτερο, το μοντέλο μας προβλέπει επιτυχία του μαθητή με πιθανότητα $\sim 77.62\%$.

```
For a student with scores 45 and 85, we predict an admission
probability of 0.7762927969028834
```

Τέλος, μπορεί να ειπωθεί ότι η παραπάνω πρόβλεψη είναι αρκετά έμπιστη, καθώς το σύστημά μας παρουσιάζει ακρίβεια πρόβλεψης:

```
Train Accuracy: 92.0
```

Θέμα 3: Εκτίμηση Παραμέτρων με Maximum Likelihood

Κατά την άσκηση αυτή, θα πραγματοποιηθεί εκτίμηση των παραμέτρων μέσης τιμής και της συνδιασποράς των κανονικών κατανομών που ακολουθούν τα δείγματα των κλάσεων ω_1 και ω_2 , για μονοδιάστατη και πολυδιάστατη περίπτωση.

- α') Οι απαραίτητοι υπολογισμοί για τον τελικό τύπο της εκτίμησης της μέσης τιμής και της διασποράς γίνονται παρακάτω.

Αν $\mathbf{S} = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_N \end{bmatrix}$ το διάνυσμα N ανεξάρτητων μεταξύ τους δειγμάτων της ω_1 , ορίζω $\theta = \begin{bmatrix} \mu \\ \sigma^2 \end{bmatrix}$

και έτσι η likelihood ορίζεται ως

$$p(\mathbf{S}|\theta) = p(s_1, s_2, \dots, s_N|\theta) = p(s_1|\theta)p(s_2|\theta) \dots p(s_N|\theta) = \prod_{i=1}^N p(s_i|\theta).$$

Για διευκόλυνση με τις πράξεις, ορίζω την

$$L(\theta) = \ln[p(s_i|\theta)] = \ln \prod_{i=1}^N p(s_i|\theta) = \sum_{i=1}^N \ln[p(s_i|\theta)] = -\sum_{i=1}^N [\ln(\sqrt{2\pi}\sigma) + \frac{(s_i - \mu)^2}{2\sigma^2}].$$

- Για την εύρεση της μ :

$$\frac{\partial L(\theta)}{\partial \mu} = 0 \Rightarrow \sum_{i=1}^N \frac{(s_i - \mu)}{\sigma^2} = 0 \Rightarrow \hat{\mu}_{ML} = \frac{1}{N} \sum_{i=1}^N s_i$$

- Για την εύρεση της σ^2 :

$$\frac{\partial L(\theta)}{\partial \sigma^2} = 0 \Rightarrow \sum_{i=1}^N \frac{(s_i - \mu)^2}{2\sigma^4} - \sum_{i=1}^N \frac{1}{2\sigma_i^2} = 0 \Rightarrow \hat{\sigma}_{ML}^2 = \frac{1}{N} \sum_{i=1}^N (s_i - \mu)^2$$

Γράφοντας τον κώδικα που υλοποιεί τα παραπάνω αποτελέσματα, και εκτελώντας τον, λαμβάνουμε τα εξής στην κονσόλα:

```
The MLE for the mean of w1 given by x1 is: -0.0709
The MLE for the covariance of w1 given by x1 is: 0.9062
-----
The MLE for the mean of w1 given by x2 is: -0.6047
The MLE for the covariance of w1 given by x2 is: 4.2007
-----
The MLE for the mean of w1 given by x3 is: -0.9110
The MLE for the covariance of w1 given by x3 is: 4.5419
```

Για τις υπό αναζήτηση παραμέτρους πολυδιάστατων κανονικών κατανομών, ορίζω $\theta = \begin{bmatrix} \mu \\ \Sigma \end{bmatrix}$

και έτσι, αν $\mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \\ \vdots \\ \mathbf{s}_N \end{bmatrix}$ το διάνυσμα (πολυδιάστατων) και ανεξάρτητων N δειγμάτων,

μπορώ, όπως και πριν να βρω την likelihood ως

$$p(\mathbf{S}|\theta) = \prod_{i=1}^N p(s_i|\theta) \text{ και να ορίσω την}$$

$$L(\theta) = \ln[p(s_i|\theta)] = -\frac{N}{2} \ln[(2\pi)^d |\Sigma|] - \frac{1}{2} \sum_{i=1}^N (\mathbf{s}_i - \mu)^T \Sigma^{-1} (\mathbf{s}_i - \mu)$$

Για την εύρεση των $\hat{\mu}_{ML}$ και $\hat{\Sigma}_{ML}$, αρκεί να παραγωγίσω την παραπάνω έκφραση ως προς και τις δύο παραμέτρους και να θέσω ίσο με το 0.

- Για την $\hat{\mu}_{\text{ML}}$:

$$\frac{\partial L(\theta)}{\partial \mu} = 0 \Rightarrow -\frac{1}{2} \sum_{i=1}^N 2(\mathbf{s}_i - \mu)^T \Sigma^{-1} = 0 \Rightarrow \hat{\mu}_{\text{ML}} = \frac{1}{N} \sum_{i=1}^N \mathbf{s}_i$$

όπου χρησιμοποιήθηκε η ιδιότητα $\frac{\partial(\mathbf{x}^T \mathbf{B} \mathbf{x})}{\partial \mathbf{x}} = 2\mathbf{B} \mathbf{x}$

- Για την $\hat{\Sigma}_{\text{ML}}$:

$$\frac{\partial L(\theta)}{\partial \Sigma} = 0 \Rightarrow -\frac{N}{2} \frac{\partial}{\partial \Sigma} \ln |\Sigma| - \frac{1}{2} \sum_{i=1}^N \frac{\partial}{\partial \Sigma} \text{tr}[(\mathbf{s}_i - \mu)^T \Sigma^{-1} (\mathbf{s}_i - \mu)] = 0 \stackrel{(**), (***)}{\Rightarrow}$$

$$-\frac{N}{2} \Sigma^{-1} - \frac{1}{2} \sum_{i=1}^N (-\Sigma^{-1})(\mathbf{s}_i - \mu)(\mathbf{s}_i - \mu)^T \Sigma^{-1} = 0 \Rightarrow$$

$$-N\Sigma + \sum_{i=1}^N (\mathbf{s}_i - \mu)(\mathbf{s}_i - \mu)^T = 0 \Rightarrow \Sigma = \frac{1}{N} \sum_{i=1}^N (\mathbf{s}_i - \mu)(\mathbf{s}_i - \mu)^T$$

όπου στο (*) έγινε χρήση του $x^T A x = \text{tr}[x^T A x]$,

στο (**) του $\frac{\partial \ln |X|}{\partial X} = (X^{-1})^T = (X^T)^{-1}$

και στο (***) του $\frac{\partial}{\partial X}(x^T X x) = x x^T$

β') Χρησιμοποιώντας τα παραπάνω αποτελέσματα και ομαδοποιώντας τις στήλες χαρακτηριστικών της ω_1 στις 3 πιθανές δυνάδες που δύναται να σχηματιστούν, εκτελούμε το Python κώδικα, που μας δίνει:

```
The MLE for the mean of w1 given by x1,x2 is:
[-0.0709 -0.6047]
The MLE for the covariance matrix of w1 given by x1,x2 is:
[[0.9062 0.5678]
 [0.5678 4.2007]]
-----
The MLE for the mean of w1 given by x2,x3 is:
[-0.6047 -0.911 ]
The MLE for the covariance matrix of w1 given by x2,x3 is:
[[4.2007 0.7337]
 [0.7337 4.5419]]
-----
The MLE for the mean of w1 given by x3,x0 is:
[-0.911 -0.0709]
The MLE for the covariance matrix of w1 given by x3,x0 is:
[[4.5419 0.3941]
 [0.3941 0.9062]]
-----
```

γ') Ακολουθώντας ίδια λογική για την τρισδιάστατη περίπτωση χαρακτηριστικών της ω_1 , εκτελούμε τον κώδικα ο οποίος μας δίνει τα κάτωθι αποτελέσματα:

```
The MLE for the mean of w1 given by its 3D samples is:
[-0.0709 -0.6047 -0.911 ]
The MLE for the covariance matrix of w1 given by its 3D samples
is:
[[0.9062 0.5678 0.3941]
 [0.5678 4.2007 0.7337]
 [0.3941 0.7337 4.5419]]
```

δ') Για την ειδική περίπτωση κατά την οποία ο πίνακας συνδιασποράς είναι διαγώνιος, θα χρειαστεί να κάνουμε κάποιους νέους υπολογισμούς για να εξάγουμε τους τύπους εύρεσης των παραμέτρων $\hat{\mu}_{\text{ML}}$ και $\hat{\Sigma}_{\text{ML}}$.

Αρχικά, είναι προφανές ότι η διαγωνιότητα του πίνακα συνδιασποράς δεν επηρεάζει με κάποιον τρόπο τη μορφή που θα έχει ο τύπος εύρεσης της $\hat{\mu}_{\text{ML}}$, πόσο μάλλον όταν παραγωγίζοντας στις προηγούμενες αποδείξεις ως προς $\hat{\mu}_{\text{ML}}$, οι όροι με τον πίνακα συνδιασποράς εξαφανίζονται (και οι παράγοντες με αυτόν μέσα δεν επηρεάζουν τον όρο που επιβιώνει).

Έτσι, ακόμη μια φορά έχουμε ότι
$$\hat{\mu}_{\text{ML}} = \frac{1}{N} \sum_{i=1}^N \mathbf{s}_i.$$

Για την εύρεση της παραμέτρου $\hat{\Sigma}_{\text{ML}}$, αρκεί να σκεφτούμε ότι $\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_d^2 \end{bmatrix}$ και

επομένως αρκεί να βρεθούν οι επιμέρους μονοδιάστατες διασπορές για κάθε μια από τις 3 διαστάσεις, όπως υπολογίστηκαν στο (α), και να αντικατασταθούν στον παραπάνω πίνακα.

Γράφοντας και εκτελώντας τον αντίστοιχο κώδικα, λαμβάνουμε:

The MLE for the mean of w1 given by x3 is:

```
[-0.1104  0.4299  0.0037]
```

The MLE for the covariance of w1 given by x3 is:

```
[[0.0544  0.      0.      ]
 [0.      0.046   0.      ]
 [0.      0.      0.0073]]
```

ε') Εύκολα παρατηρεί κανείς ότι, οι 'συνιστώσες' των πολυδιάστατων μέσων τιμών δεν είναι παρά οι μέσες τιμές που ανακτά κανείς κατά τους υπολογισμούς σε μονοδιάστατη περίπτωση μελέτης. Παράλληλα, σε ότι αφορά τους πίνακες συνδιασποράς, τα διαγώνια στοιχεία τους είναι ίσα με τις διασπορές που υπολογίζονται στη μονοδιάστατη περίπτωση, και τα μη διαγώνια στοιχεία είναι μη μηδενικά, πράγμα ενδεικτικό εξάρτησης μεταξύ των συνιστοσών του τυχαίου διανύσματος. Δηλαδή, η υπόθεση που κάναμε περί ανεξαρτησίας των δειγμάτων κάθε διάστασης για την εκτίμηση μέγιστης πιθανοφάνειας δεν είναι απόλυτα ακριβής. Ωστόσο, λόγω των σχεδόν μηδενικών τιμών (τείνουν προς το 0) που υπάρχουν στα μη διαγώνια στοιχεία των πινάκων συνδιασποράς, η παραπάνω παραδοχή εμπεριέχει μικρό σφάλμα κατά την εκτίμηση των παραμέτρων κάθε κατανομής.

Θέμα 4:Ομαδοποίηση (Clustering) με K-means και GMM

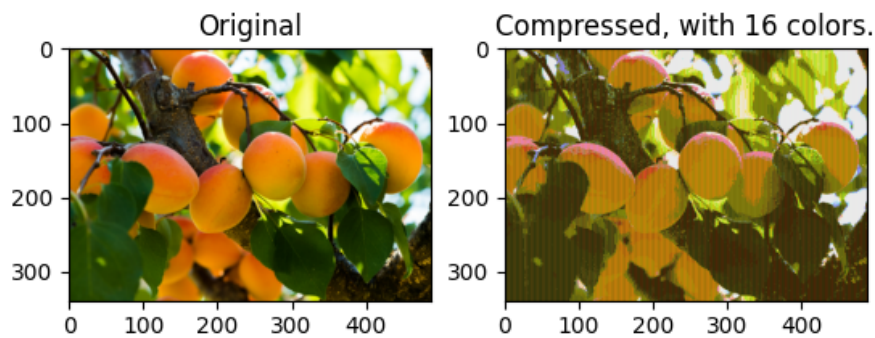
Σε αυτή την άσκηση θα υλοποιηθεί ο K-means clustering αλγόριθμος και θα χρησιμοποιηθεί για να συμπίεστεί μια εικόνα.

α') Υλοποιήθηκε η 'findClosestCentroids()', η οποία ψάχνει αναδρομικά το κοντινότερο κεντροειδές για κάθε $x^{(i)}$ της εικόνας.

β') Στην συνέχεια, εφόσον έχει βρεθεί το κάθε σύνολο C_k που περιέχει τα στοιχεία $x^{(i)}$ που αντιστοιχίζονται στο κεντροειδές k , στη συνάρτηση 'computeCentroids()' υπολογίζεται το κεντροειδές της κάθε κλάσης σύμφωνα με τον τύπο:

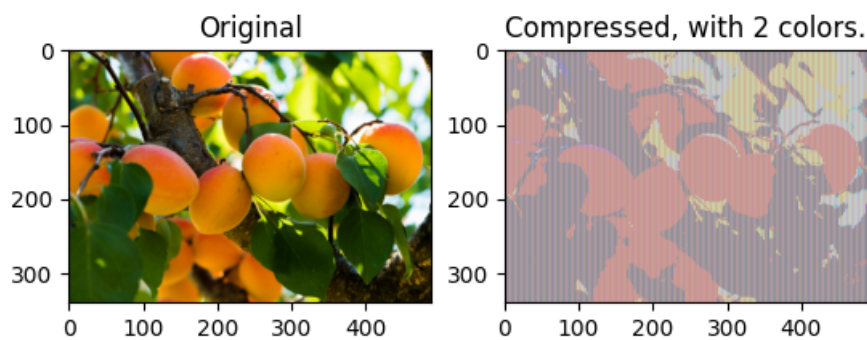
$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x^{(i)}$$

γ') Με την εκτέλεση του κώδικα εμφανίζεται η παρακάτω εικόνα:

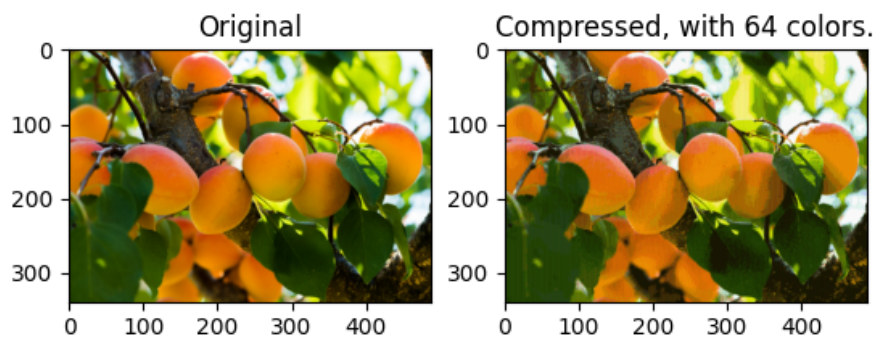


Σχήμα 7: Η αυθεντική Fruit.png και η ανεκτημένη για 16 χρώματα

Εκτελώντας δοκιμές για λιγότερα χρώματα από 16 και εν συνεχεία για περισσότερα, λαμβάνουμε αντίστοιχα τις παρακάτω ανεκτημένες εικόνες:



Σχήμα 8: Η αυθεντική Fruit.png και η ανεκτημένη για 2 χρώματα



Σχήμα 9: Η αυθεντική Fruit.png και η ανεκτημένη για 64 χρώματα

Παρατηρούμε ότι με την μείωση των κεντροειδών σε $K=2$ η ποιότητα της ανεκτημένης εικόνας σε σύγκριση με την περίπτωση που $K=16$ είναι υποδιαίστερη, υπό την έννοια ότι τα χρώματα εμφανίζονται πιο θολά, δηλαδή υπάρχει μικρότερη διαχωριστικότητα μεταξύ τους. Το παραπάνω είναι ενδεικτικό της μείωσης των κλάσεων διαχωρισμού (clusters), δηλαδή της λεπτομέρειας που μπορούμε να διακρίνουμε στην εικόνα.

Αντίστοιχα, με την αύξηση του αριθμού των κεντροειδών $K=64$ η ποιότητα της ανεκτημένης εικόνας αυξάνεται συγκριτικά με την περίπτωση όπου $K=16$ και αυτό γιατί (σε αναλογία με τα προηγούμενα) ο διαχωρισμός σε μεγαλύτερο αριθμό κλάσεων (clusters) ισοδυναμεί με την ανάπτυξη μεγαλύτερης λεπτομέρειας στην εικόνα.

Θέμα 5: Νευρωνικά Δίκτυα

Η άσκηση που επιλέχθηκε για το κομμάτι των νευρωνικών δικτύων είναι η 5Α.

Θέμα 5α: Υλοποίηση ενός απλού νευρωνικού δικτύου.

Στόχος είναι να υλοποιηθεί και να εκπαιδευτεί ένα νευρωνικό δίκτυο χωρίς την χρήση έτοιμων προγραμμάτων/βιβλιοθηκών. Η υλοποίηση θα γίνει σε python χρησιμοποιώντας τη βιβλιοθήκη numpy.

Μέρος Α:

α') Δίνεται η σχέση:

$$\hat{y}^{(i)} = f(z^{(i)})$$

όπου $z^{(i)} = x^{(i)}W + b$ και $f(z) = \frac{1}{1+e^{-z}}$, πρέπει να δείξουμε ότι:

$$J(Y, \hat{Y}; W, b) = \frac{1}{B} \sum_i \left(z^{(i)} - z^{(i)} y^{(i)} + \ln(1 + e^{-z^{(i)}}) \right)$$

Συνάρτηση απώλειας cross-entropy:

$$J(y^{(i)}, \hat{y}^{(i)}) = -y^{(i)} \ln(\hat{y}^{(i)}) - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})$$

Αντικαθιστώντας $\hat{y}^{(i)} = f(z^{(i)})$:

$$J(y^{(i)}, f(z^{(i)})) = -y^{(i)} \ln(f(z^{(i)})) - (1 - y^{(i)}) \ln(1 - f(z^{(i)}))$$

Εφόσον $f(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$, έχουμε: $1 - f(z^{(i)}) = \frac{e^{-z^{(i)}}}{1+e^{-z^{(i)}}}$.

Άρα:

$$\ln(f(z^{(i)})) = \ln\left(\frac{1}{1+e^{-z^{(i)}}}\right) = -\ln(1+e^{-z^{(i)}})$$

$$\ln(1 - f(z^{(i)})) = \ln\left(\frac{e^{-z^{(i)}}}{1+e^{-z^{(i)}}}\right) = -z^{(i)} - \ln(1+e^{-z^{(i)}})$$

Αντικατάσταση με τη συνάρτηση απώλειας (cross-entropy):

$$J(y^{(i)}, f(z^{(i)})) = -y^{(i)}(-\ln(1+e^{-z^{(i)}})) - (1 - y^{(i)})(-z^{(i)} - \ln(1+e^{-z^{(i)}})) \Rightarrow$$

$$J(y^{(i)}, f(z^{(i)})) = y^{(i)} \ln(1+e^{-z^{(i)}}) + (1 - y^{(i)})(z^{(i)} + \ln(1+e^{-z^{(i)}}))$$

$$\begin{aligned}
&= y^{(i)} \ln(1 + e^{-z^{(i)}}) + z^{(i)} + \ln(1 + e^{-z^{(i)}}) - y^{(i)} z^{(i)} - y^{(i)} \ln(1 + e^{-z^{(i)}}) \\
&= z^{(i)} - z^{(i)} y^{(i)} + \ln(1 + e^{-z^{(i)}})
\end{aligned}$$

Συνεπώς:

$$J(Y, \hat{Y}; W, b) = \frac{1}{B} \sum_i \left(z^{(i)} - z^{(i)} y^{(i)} + \ln(1 + e^{-z^{(i)}}) \right)$$

β') Πρέπει να δείξουμε ότι:

$$\frac{\partial J}{\partial z^{(i)}} = -y^{(i)} + \hat{y}^{(i)}$$

Από το πρώτο ερώτημα:

$$J = z^{(i)} - z^{(i)} y^{(i)} + \ln(1 + e^{-z^{(i)}})$$

Παραγωγίζω:

$$\frac{\partial J}{\partial z^{(i)}} = 1 - y^{(i)} + \frac{\partial}{\partial z^{(i)}} \ln(1 + e^{-z^{(i)}})$$

Η παράγωγος του $\ln(1 + e^{-z^{(i)}})$ είναι:

$$\frac{\partial}{\partial z^{(i)}} \ln(1 + e^{-z^{(i)}}) = \frac{1}{1 + e^{-z^{(i)}}} \cdot (-e^{-z^{(i)}}) = -\frac{e^{-z^{(i)}}}{1 + e^{-z^{(i)}}}$$

Εφόσον $\hat{y}^{(i)} = \frac{1}{1 + e^{-z^{(i)}}}$, προκύπτει:

$$\frac{e^{-z^{(i)}}}{1 + e^{-z^{(i)}}} = 1 - \hat{y}^{(i)}$$

Συνεπώς:

$$\frac{\partial J}{\partial z^{(i)}} = 1 - y^{(i)} - (1 - \hat{y}^{(i)}) = -y^{(i)} + \hat{y}^{(i)}$$

γ') Ξέρουμε ότι $z^{(i)} = x^{(i)}W + b$ και $\hat{y}^{(i)} = f(z^{(i)})$:

$$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial W} = (\hat{y}^{(i)} - y^{(i)}) \cdot \frac{\partial z^{(i)}}{\partial W}$$

Εφόσον $z^{(i)} = x^{(i)}W + b$, έχουμε:

$$\frac{\partial z^{(i)}}{\partial W} = x^{(i)T}$$

Συνεπώς:

$$\frac{\partial J}{\partial W} = \sum_i (\hat{y}^{(i)} - y^{(i)}) x^{(i)T}$$

Αντίστοιχα, για το b :

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial b} = (\hat{y}^{(i)} - y^{(i)}) \cdot 1 = \sum_i (\hat{y}^{(i)} - y^{(i)})$$

δ') Για το back-propagation σε ένα νευρωνικό δίκτυο με πολλαπλά επίπεδα (στην περίπτωση μας 3 layers), χρησιμοποιώντας την cross-entropy ως συνάρτηση απώλειας και την λογιστική συνάρτηση ως συνάρτηση ενεργοποίησης, τα βήματα ενημέρωσης για τα βάρη και τις προκαταλήψεις είναι:

1. Forward Propagation

Οι είσοδοι περνούν από το πρώτο επίπεδο και παράγουν την έξοδο, αντίστοιχα και για τα 3 επίπεδα, ενώ ταυτόχρονα γίνεται και ο υπολογισμός του σφάλματος E με την χρήση της συνάρτησης cross-entropy.

2. Back Propagation

Υπολογίζονται τα σφάλματα στην έξοδο κάθε επιπέδου.

3. Ενημέρωση των βαρών και των bias

Τα βάρη ενημερώνονται με βάση το Gradient Descent:

$$W \leftarrow W - \rho \frac{\partial J}{\partial W}$$

Τα bias ενημερώνονται αντίστοιχα:

$$b \leftarrow b - \rho \frac{\partial J}{\partial b}$$

όπου ρ είναι ο ρυθμός μάθησης (learning-rate).

Μέρος B:

Αρχικά, πριν οποιαδήποτε αλλαγή στις παραμέτρους του νευρωνικού δικτύου η εκτέλεση του κώδικα μας δίνει τα παρακάτω (default) αποτελέσματα:

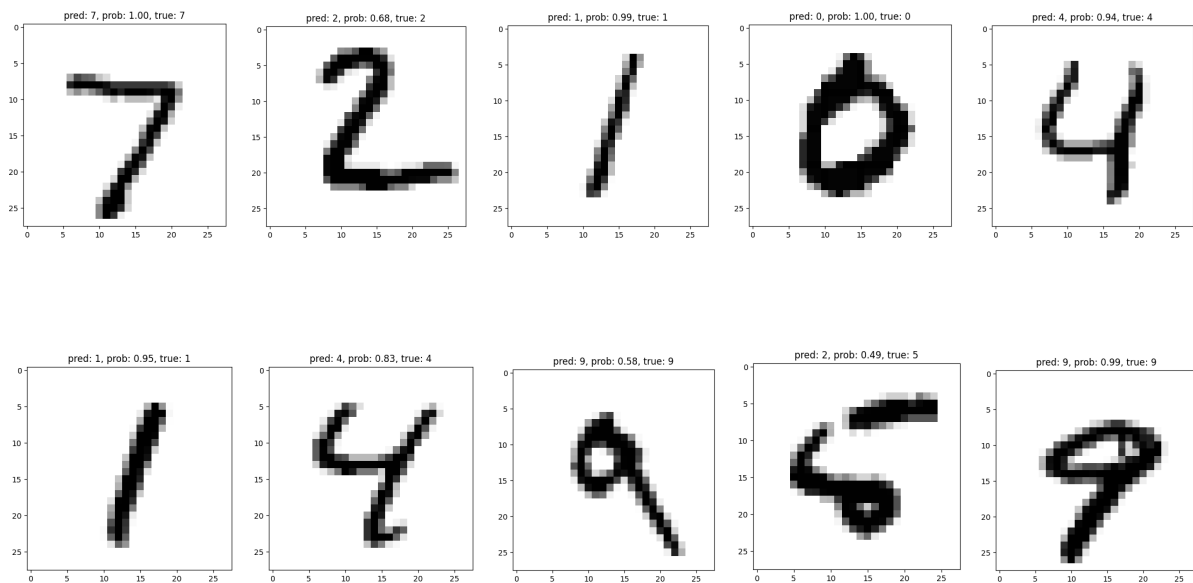
:

```
97/100, error=0.0358803095786178
98/100, error=0.010448722904515283
99/100, error=0.025566563588244634
100/100, error=0.05635487374965777
pred: 7      true: 7
pred: 0      true: 2
pred: 1      true: 1
pred: 0      true: 0
pred: 4      true: 4
pred: 1      true: 1
pred: 4      true: 4
pred: 9      true: 9
pred: 2      true: 5
pred: 9      true: 9
pred: 0      true: 0
pred: 2      true: 6
pred: 9      true: 9
pred: 0      true: 0
pred: 1      true: 1
pred: 5      true: 5
```

```

pred: 9      true: 9
pred: 7      true: 7
pred: 3      true: 3
pred: 4      true: 4
ratio: 0.85
mse: 0.0145

```



Σχήμα 11: Οι προβλέψεις του Ν.Δ για τα 10 εικονιζόμενα ψηφία που δόθηκαν για evaluation

- Αλλάζοντας το learning rate και με σταθερά τα epochs=100 και batch size=128 λαμβάνουμε τα εξής αποτελέσματα:

– Για learning rate=0.0001:

:

97/100, error=1.4036768137840987

98/100, error=1.4945557197203982

99/100, error=1.330420432831114

100/100, error=1.261234218458012

pred: 6 true: 7

pred: 8 true: 2

pred: 2 true: 1

pred: 6 true: 0

pred: 2 true: 4

pred: 5 true: 1

pred: 0 true: 4

pred: 2 true: 9

pred: 8 true: 5

pred: 1 true: 9

pred: 2 true: 0

pred: 2 true: 6

pred: 5 true: 9

pred: 2 true: 0

```

pred: 2      true: 1
pred: 2      true: 5
pred: 2      true: 9
pred: 6      true: 7
pred: 5      true: 3
pred: 7      true: 4
ratio: 0.15

```

– Για learning rate=0.75:

```

:
97/100, error=0.012463615424033054
98/100, error=0.032888733799156994
99/100, error=0.015042399634508364
100/100, error=0.06105846441417374
pred: 7      true: 7
pred: 6      true: 2
pred: 1      true: 1
pred: 0      true: 0
pred: 4      true: 4
pred: 1      true: 1
pred: 4      true: 4
pred: 4      true: 9
pred: 6      true: 5
pred: 7      true: 9
pred: 0      true: 0
pred: 6      true: 6
pred: 9      true: 9
pred: 0      true: 0
pred: 1      true: 1
pred: 5      true: 5
pred: 9      true: 9
pred: 7      true: 7
pred: 5      true: 3
pred: 4      true: 4
ratio: 0.80

```

Για υπερβολικά μικρό learning rate το error που προέκυψε παρατηρείται ότι είναι αρκετά μεγαλύτερο από την default περίπτωση και αυτό γιατί το πολύ μικρό βήμα (step size) που μας παρέχει το learning rate ‘έξαναγκάζει’ τον αλγόριθμο να συγκλίνει σε κάποιο τοπικό ελάχιστο και όχι ολικό ελάχιστο, με αποτέλεσμα να μην προκύπτει η βέλτιστη λύση.

Αντίθετα, για λογικά μεγαλύτερο learning rate από το default προκύπτουν τιμές σφάλματος ανάλογα καλές με της default περίπτωσης και αυτό γιατί μας παρέχει ένα βήμα το οποίο είναι εύλογα μεγάλο για να επιτύχουμε βέλτιστη σύγκλιση.

- Αλλάζοντας τον αριθμό των epochs και με σταθερά τα learning rate=0.1 και batch size=128 λαμβάνουμε τα εξής αποτελέσματα:

– Για epochs=1:

```

1/1, error=167.03942169203373
pred: 3      true: 7
pred: 5      true: 2

```

```

pred: 7      true: 1
pred: 0      true: 0
pred: 1      true: 4
pred: 9      true: 1
pred: 5      true: 4
pred: 9      true: 9
pred: 3      true: 5
pred: 7      true: 9
pred: 3      true: 0
pred: 1      true: 6
pred: 9      true: 9
pred: 0      true: 0
pred: 9      true: 1
pred: 3      true: 5
pred: 1      true: 9
pred: 9      true: 7
pred: 9      true: 3
pred: 3      true: 4
ratio: 0.15

```

– $\Gamma\alpha$ epochs=250:

```

:
245/250, error=0.11143267464477989
246/250, error=0.1486927845753224
247/250, error=0.12133822302747803
248/250, error=0.132058766263464
249/250, error=0.1714613384000955
250/250, error=0.14871427204464668
pred: 7      true: 7
pred: 5      true: 2
pred: 1      true: 1
pred: 0      true: 0
pred: 4      true: 4
pred: 1      true: 1
pred: 4      true: 4
pred: 2      true: 9
pred: 2      true: 5
pred: 4      true: 9
pred: 0      true: 0
pred: 4      true: 6
pred: 7      true: 9
pred: 0      true: 0
pred: 1      true: 1
pred: 5      true: 5
pred: 4      true: 9
pred: 7      true: 7
pred: 5      true: 3
pred: 4      true: 4
ratio: 0.85

```

Δεδομένου ότι τα epochs είναι ουσιαστικά ο αριθμός των φορών που περνάει κάθε dataset τουλάχιστον μία φορά με forward pass και backward pass καταλαβαίνουμε ότι περνώντας τα δεδομένα μόλις μια φορά από το νευρωνικό δίκτυο αυτό δεν προλαβαίνει να κάνει update τα βάρη του ικανοποιητικό αριθμό φορών με αποτέλεσμα να λαμβάνουμε ένα πολύ κακής ποιότητας αποτέλεσμα (υψηλό error).

Αντίθετα, για μεγάλο αριθμό epochs(250) τα forward και backward pass λαμβάνουν χώρα πολλές φορές με αποτέλεσμα τα βάρη να ανανεώνονται εξίσου πολλές φορές και τελικά να τείνουν σε μία βέλτιστη τιμή.

- Αλλάζοντας τον αριθμό των batch size και με σταθερά τα learning rate=0.1 και epochs=100 λαμβάνουμε τα εξής αποτελέσματα:

– Για batch size=512:

:

```
95/100, error=1.2870013441743164
96/100, error=1.8102446639910876
97/100, error=1.368642972285067
98/100, error=1.4008809852517679
99/100, error=1.4941714892012452
100/100, error=1.6797685682761334
```

```
pred: 7      true: 7
pred: 3      true: 2
pred: 1      true: 1
pred: 0      true: 0
pred: 6      true: 4
pred: 1      true: 1
pred: 4      true: 4
pred: 7      true: 9
pred: 6      true: 5
pred: 7      true: 9
pred: 0      true: 0
pred: 2      true: 6
pred: 9      true: 9
pred: 0      true: 0
pred: 1      true: 1
pred: 5      true: 5
pred: 7      true: 9
pred: 7      true: 7
pred: 3      true: 3
pred: 4      true: 4
ratio: 0.85
```

– Για batch size=16:

:

```
95/100, error=0.00016863383803980428
96/100, error=0.00013863274555157056
97/100, error=0.00011375220531640813
98/100, error=0.00012158328351577442
99/100, error=7.910446339486506e-05
100/100, error=8.585036688359474e-05
pred: 7      true: 7
```

```

pred: 5      true: 2
pred: 1      true: 1
pred: 0      true: 0
pred: 4      true: 4
pred: 1      true: 1
pred: 4      true: 4
pred: 9      true: 9
pred: 6      true: 5
pred: 9      true: 9
pred: 0      true: 0
pred: 2      true: 6
pred: 9      true: 9
pred: 0      true: 0
pred: 1      true: 1
pred: 4      true: 5
pred: 9      true: 9
pred: 7      true: 7
pred: 3      true: 3
pred: 4      true: 4
ratio: 0.75

```

Το batch size αναφέρεται στον αριθμό των δειγμάτων που λαμβάνουμε υπόψιν από το dataset για μια επανάληψη εκπαίδευσης του δικτύου. Συνεπώς, είναι ευνόητο ότι για μεγαλύτερο batch size λιγότερες επαναλήψεις του αλγορίθμου θα λαμβάνουν χώρα για την εκπαίδευση του δικτύου άρα και λιγότερα updates των βαρών του.

Απεναντίας, για εύλογα μικρό batch size τα πακέτα εκπαίδευσης που λαμβάνουμε ανά επανάληψη του αλγορίθμου είναι πιο μικρά οδηγώντας έτσι σε περισσότερες επαναλήψεις εκπαίδευσης και τελικά αναβάθμισης των βαρών.

2. Αλλάζοντας την συνάρτηση ενεργοποίησης του πρώτου layer σε tanh και συνάρτηση κόστους την MSE:

```

:
97/100, error=0.0019258754205171437
98/100, error=0.002218732225039693
99/100, error=0.0018343196481614924
100/100, error=0.0017786095717649232
pred: 7      true: 7
pred: 2      true: 2
pred: 1      true: 1
pred: 0      true: 0
pred: 7      true: 4
pred: 1      true: 1
pred: 8      true: 4
pred: 9      true: 9
pred: 6      true: 5
pred: 7      true: 9
pred: 0      true: 0
pred: 1      true: 6
pred: 9      true: 9
pred: 0      true: 0

```

```

pred: 1      true: 1
pred: 0      true: 5
pred: 9      true: 9
pred: 7      true: 7
pred: 8      true: 3
pred: 8      true: 4
ratio: 0.60
mse: 0.0375

```

Παρατηρούμε ότι σε σύγκριση με την default περίπτωση το MSE είναι λίγο μεγαλύτερο, πράγμα το οποίο αρχικά φαίνεται οξύμωρο καθώς αναμένουμε ότι ελαχιστοποιώντας το κόστος ως προς την συνάρτηση MSE (και όχι πλέον την cross entropy) το τελικό αποτέλεσμα του MSE κόστους θα ήταν μικρότερο συγκριτικά με πριν.

Ωστόσο, λόγω του ότι η cross entropy cost function διαχειρίζεται με διαφορετική "επιθετικότητα" διαφορετικά error στην πρόβλεψη ενώ η MSE αντιμετωπίζει όλα τα error το ίδιο, αλλά και του ότι η συνάρτηση ενεργοποίησης του τελευταίου επιπέδου είναι η softmax, η MSE καθίσταται λιγότερο κατάλληλη της cross entropy για το συγκεκριμένο classification πρόβλημα.

Τέλος, μικρό ρόλο παίζει και η επιλογή της tanh ως συνάρτησης ενεργοποίησης στην ακρίβεια του νευρωνικού δικτύου και αυτό γιατί η ίδια αποτελεί μια προσέγγιση της sigmoid με την οποία την αντικαταστήσαμε.

3. Για την αλλαγή σε δίκτυο παραπάνω κόμβων εισόδου και hidden layers έγιναν οι παρακάτω αλλαγές στον κώδικα:

```

network = [
    Dense(28 * 28, 128), # Increased number of nodes
    Sigmoid(),
    Dense(128, 64), # Added more hidden layers
    Sigmoid(),
    Dense(64, 32), # Added another hidden layer
    Sigmoid(),
    Dense(32, 10),
    Softmax()
]

```

και προέκυψαν τα εξής αποτελέσματα:

```

:
97/100, error=0.026763727030112035
98/100, error=0.01869519257263068
99/100, error=0.018366935817254355
100/100, error=0.006373012403891544
pred: 7      true: 7
pred: 5      true: 2
pred: 1      true: 1
pred: 0      true: 0
pred: 4      true: 4
pred: 1      true: 1
pred: 4      true: 4
pred: 9      true: 9

```

```
pred: 2      true: 5
pred: 7      true: 9
pred: 0      true: 0
pred: 2      true: 6
pred: 9      true: 9
pred: 0      true: 0
pred: 1      true: 1
pred: 7      true: 5
pred: 9      true: 9
pred: 7      true: 7
pred: 6      true: 3
pred: 4      true: 4
ratio: 0.70
```

Είναι εμφανές ότι η προσθήκη περισσότερων 'φίλτρων' (κόμβων και κρυφών επιπέδων) για τα δεδομένα που περνάνε μέσα απο το νευρωνικό δίκτυο, οδήγησε σε μικρότερες τιμές σφάλματος.